

Kolokwium 2 - Java

Imię:

Nazwisko:

Na napisanie testu jest 85 minut. Każde poprawnie rozwiązane zadanie to jeden punkt. Każde niepoprawnie rozwiązane zadanie to zero punktów (dotyczy to również zadań z wielokrotnym wyborem)

1. Adnotacja zdefiniowana w Javie może
 - a. odnosić się tylko do metod.
 - b. być wymazywana w trakcie kompilacji.
 - c. być dostępna poprzez mechanizm refleksji.
 - d. być niedostępna poprzez mechanizm refleksji.
2. Jaki jest wynik działania następującego programu (nie zwracać uwagi na importy)?

```
class Test implements Runnable {  
    public void run() {  
        try {while(true) System.out.println("X");  
        } finally {System.out.println("Koniec");}  
    }  
}  
  
public class Zadanie{  
    public static void main(String[] args) throws  
    InterruptedException {  
        Thread t = new Thread(new Test());  
        t.setDaemon(true);  
        t.start();  
        TimeUnit.SECONDS.sleep(1);  
    }  
}
```

- a. Będzie się wypisywać X w nieskończoność.
 - b. Przez sekundę będzie wypisywać się X, a potem wypisze się Koniec i program się zakończy.
 - c. Przez sekundę będzie wypisywać się X, a potem program zakończy się bez błędu.
 - d. Przez sekundę będzie wypisywać się X, a potem zostanie rzucony wyjątek i program się zakończy.
3. Jaki jest wynik uruchomienia następującego fragmentu kodu?

```
List<Integer> a = new ArrayList<Integer>();  
List<String> b = new LinkedList<String>();  
Set<Integer> c = new HashSet<Integer>();  
Set<String> d = new TreeSet<String>();  
System.out.println(a.equals(b) + ", " + c.equals(d) + ", " + a.equals(c) + "  
, " + b.equals(d) );
```

- a. true,true,true,true
 - b. true,true,false,false
 - c. false,false,true,true
 - d. false,false,false,false
4. Jeśli dana klasa ma być serializowana, to musi implementować interfejs Serializable oraz nadpisać metody:
- a. toString
 - b. writeObject
 - c. readObject
 - d. Żadna z powyższych.
5. Czy w programie wielowątkowym, w którym każdy wątek tworzy jedną instancję poniższej klasy metoda getId() zawsze zwróci inną wartość dla różnych obiektów?
- ```
class A {
 private static int c = 0;
 private int id = c++;
 public int getId() {return id;}
}
```
6. Jaki jest wynik kompilacji i uruchomienia następującego programu?
- ```
public enum Color implements Runnable {
    WHITE, BLACK, RED, YELLOW, BLUE;
    public void run() {System.out.println(this);}
    public static void main(String[] args) {
        for (Color c : Color.values()) new Thread(c).start();
    }
}
```
- a. Program się nie skompiluje bo klasy Enum nie mogą implementować rzadnych interfejsów.
 - b. Program się nie skompiluje bo klasa Enum nie może implementować jedynie interfejsu Runnable.
 - c. Program się skompiluje, a podczas działania zostanie utworzonych 5 wątków, które zostaną wystartowane w pętli for.
 - d. Program się skompiluje, a podczas działania zostanie utworzonych 5 wątków, które zostaną automatycznie uruchomione przed pętlą for, a następnie jeszcze raz wystartują w pętli for.
7. Które z poniższych deklaracji adnotacji są poprawne?
- a. public interface @A1 {}
 - b. public @interface @A1 {}
 - c. public @interface A1 {}
 - d. @Annotation
public interface A1 {}
8. Jakie słowo kluczowe oznacza, że dane pole nie będzie serializowane w procesie serializacji?

9. Klasa `A<T>` jest zdefiniowana następująco:

```
class A<T> { T id(T t) { return t; } }
```

zaznacz linie które skompilują i wykonają się poprawnie:

- a. `A<Integer> a = new A<Integer>(); Integer i = a.id(7);`
- b. `A<? extends Integer> b = new A<Integer>(); Integer j = b.id(7);`
- c. `A<? super Integer> c = new A<Integer>(); Object k = c.id(7);`
- d. `A<?> d = new A<Integer>(); Object l = d.id(null);`

10. Jakie sygnatury mają metody `test1` i `test2` w czasie wykonania poniższego kodu:

```
import java.util.List;
public class Test<T, S extends List<T>> {
    public void test1(T t) { }
    public void test2(S s) { }
}
```

11. Co wypisze, po wykonaniu, poniższy kod:

```
import java.nio.CharBuffer;
public class Test {
    public static void main(String[] args) {
        CharBuffer buffer = CharBuffer.allocate(128);
        buffer.put("Ala".toCharArray());
        buffer.flip();
        System.out.print("<" + buffer + ">");
        buffer.get();
        buffer.mark();
        System.out.print("<" + buffer + ">");
        buffer.get();
        System.out.print("<" + buffer + ">");
        buffer.reset();
        System.out.print("<" + buffer + ">");
    }
}
```

12. Czy poniższy kod, uruchomiony poprzez `java Test`, będzie się wykonywał w nieskończoność? Wyjaśnij dlaczego (uwzględniając prawdziwość warunku pod `if'em`).

```
import java.util.Random;
public class Test {
    public static void main(String[] args) {
        Random rG = new Random();
        int i = 0;
        while (true) {
            if ((i = rG.nextInt() % 2) == 0) {
                continue;
            } else { assert i % 2 == 1; }
        }
    }
}
```

```

    }
}

```

13. Wyjaśnij na czym polega programowanie sterowane zdarzeniami i gdzie się go używa:

.....

.....

.....

14. Czy poniższy kod się kompiluje (jeśli nie wskaż miejsce i wytłumacz dlaczego)? Jaki jest wynik jego wykonania?

```

public class Test {
    static class A<T extends Throwable> {
        A() throws T { System.out.print("A"); }
    }
    static class B extends A<Exception> {
        B() throws RuntimeException { System.out.print("B");
    }

    }
    public static void main(String[] args) throws Throwable {
new B(); }
}

```

15. Wymień dwie klasy z hierarchii obiektów służących do obsługi wyjścia bajtowego i dwie klasy służące do obsługi wyjścia tekstowego:

.....

.....

.....

.....

16. Czy poniższa klasa spełnia wymagania nałożone przez Javę na metody hashCode i equals? Odpowiedź uzasadnij.

```

public class Test {
    @Override
    public boolean equals(Object o) {
        if(o instanceof Test)
            return true;
        return false;
    }
    @Override
    public int hashCode(){return 7;}
}

```

17. Które z poniższych metod są zdefiniowane w klasie Thread ?

- a. start()
- b. wait()
- c. notify()
- d. run()
- e. terminate()

18. Jaki będzie wynik wykonania programu?

```
public class Test
{
    public static void main(String[] args)
    {
        int x = 0;
        assert (x > 0) ? "assertcja zle" : "assertcja ok" ;
        System.out.println("koniec");
    }
}
```

- a. "koniec"
- b. Błąd kompilacji.
- c. Zostanie rzucony AssertionError a na wyjściu wypisane "koniec"
- d. Zostanie rzucony AssertionError z informacją "asercja zle"

19. Zaznacz prawdziwe zdania:

- a. AWT i SWING to podstawowe biblioteki JAWY umożliwiające tworzenie graficznego interfejsu użytkownika.
- b. ActionListener to interfejs służący do odbierania informacji o wystąpieniu zdarzenia.
- c. JFrame jest podstawowym komponentem służącym do wyświetlania okna aplikacji.
- d. JLabel jest komponentem wyświetlającym przycisk.
- e. IO i NIO to podstawowe biblioteki graficzne w JAWIE umożliwiające tworzenie GUI.

20. Wskaż rezultat wykonania kodu:

```
public class Tabs {
    public static void main(String[] args){
        int [][] tab6 = {{1,2,3},{4,5,6}};
        for(int i=0; i < 2; ++i){
            for(int j=0; j < 3; ++j){
                System.out.print(tab6[i][j]);
            }
        }
    }
}
```

- a. Wypisanie: 123456
- b. Wypisanie: 654321
- c. Błąd kompilacji;
- d. Wyrzucenie wyjątku ArrayIndexOutOfBoundsException;
- e. Wypisanie: 456123
- f. Wypisanie: 321654

21. Jakie polecenie należy wstawić w miejsce ###, aby poniższy kod kompilował się i

wykonywał poprawnie?

```
public class WhileTrue {
    public static void main(String[] args){
        (new Thread(new Runnable(){

            @Override
            public void run() {
                while(true){
                    System.out.println("Hello World!");
                }
            }

        })).start();
    }
}
```

- a. run();
- b. currentThread().run();
- c. notify();
- d. start();
- e. notifyAll();

22. Zaznacz poprawne odpowiedzi na temat poniższego kodu:

```
import java.util.prefs.BackingStoreException;
import java.util.prefs.Preferences;
public class Preferencje {
    public static void main(String[] args) throws
BackingStoreException {
        Preferences p = Preferences
            .userNodeForPackage(Preferencje.class);
        if(p.get("String", null) == null){
            p.put("String", "OK");
            System.out.println("String" + " -> " +
p.get("String", null));
        }else{p.clear();}
    }
}
```

- a. Przy pierwszym uruchomieniu program nic nie wypisze.
- b. Przy drugim uruchomieniu program nic nie wypisze.
- c. Przy pierwszym uruchomieniu program wypisze: String->OK.
- d. Przy drugim uruchomieniu program wypisze: String->OK.
- e. Wystąpi błąd kompilacji.
- f. Przy drugim uruchomieniu zostanie rzucony wyjątek.

23. Jaki będzie rezultat wykonania poniższego kodu?

```

import java.util.ArrayList;
import java.util.Collection;
public class Kolekcja {
    public static void main(String[] args){
        Collection<Integer> c = new ArrayList<Integer>();
        for(int i = 99; i <= 102; i++){
            c.add(i);
        }
        for(Integer i : c){
            System.out.println("OK " + i);
            if(i.equals(100)) c.remove(i);
        }
    }
}

```

- a. Błąd kompilacji.
- b. Wypisanie:
 - OK 102
 - OK 101
 - OK 100
 - OK 99
- c. Zostanie rzucony wyjątek ConcurrentModificationException.
- d. Wypisanie:
 - OK 99
 - OK 100
 - OK 101
 - OK 102

24. Co wypisze poniższy kod?

```

enum E { ONE, TWO }
public class A {
    public static void main(String[] args)
    {      System.out.println(E.class);      }
}

```

- a. "enum E"
- b. "class Enum"
- c. "class E"

25. Co wypisze poniższy kod?

```

public enum Kolor {
    CZERWONY(false),
    ZIELONY(true),
    NIEBIESKI(true),
    CZARNY(false);
}

```

```

boolean ladny;

private Kolor(boolean czyLadny) {
    ladny = czyLadny;
}

@Override
public String toString() {
    return super.toString().toLowerCase();
}

public static void main(String[] args) {
    for(Kolor kolor: Kolor.values()) {
        System.out.println("Kolor "+kolor+" jest "+
            (kolor.ladny ? "ladny" : "brzydki"));
    }
}

```

```

.....
.....
.....
.....
.....
.....
.....
.....

```

26. Przeanalizuj uważnie następujący kod:

```

class MyRunnable implements Runnable {
    public static int a = 0;
    synchronized void increment() { ++a; }
    public void run() {
        for (int i = 0; i < 10; ++i)
            increment();
    }
}

public class A {
    public static void main(String[] args)
        throws InterruptedException {
        Thread[] threads = new Thread[10];
        for (int i = 0; i < 10; ++i) {
            threads[i] = new Thread(new MyRunnable());
            threads[i].start();
        }
    }
}

```


Efektem jego wykonania

27. Omów składnię bloku synchronizowanego w Javie i wyjaśnij do czego on służy.

[illegible]

Który z poniższych kodów skompiluje się bez błędów?

29. Która z poniższych definicji metody ogólnej $\neq$ jest poprawna?

- a. `public static T f<T>(T t) { return t; }`
- b. `public static T <T> f(T t) { return t; }`
- c. `public static <T> T f(T t) { return t; }`
- d. `public <T> static T f(T t) { return t; }`

30. Efektem wykonania następującego kodu:

```
class B {}  
class A<T extends B> {  
    public static void main(String[] args) {  
        A<B> ab = new A<B>();  
        System.out.println(ab.getClass());  
    }  
}
```

będzie

- a. wypisanie na ekran "class A"
- b. wypisanie na ekran "class A<?>"
- c. wypisanie na ekran "class A"
- d. wypisanie na ekran "class A<Object>"
- e. wyjątek